

AtallaCC: Designing a Compiler Stack for a Programmable Atalla AI Accelerator

Mikhail Golovenchits	Sahil Patel
VIP 47922	VIP 47922
mgoloven@purdue.edu	patel1701@purdue.edu
Shaunak Sarlashkar	James Erwin
VIP 47921	VIP 47920
ssarlash@purdue.edu	erwin17@purdue.edu
Aditya Hegde	
Volunteer	
hegde39@purdue.edu	

Spring 2026

The feedback for the report is to be returned to:

Mikhail Golovenchits
mgoloven@purdue.edu

This project was mentored by the following Teaching Assistants:

Timothy Hein, heint@purdue.edu
Sooraj Chetput, schetput@purdue.edu

AtallaCC: Designing a Compiler Stack for a Programmable Atalla AI Accelerator

Mikhail Golovenchits VIP 47922 mgoloven@purdue.edu	Sahil Patel VIP 47922 patel1701@purdue.edu	Shaunak Sarlashkar VIP 47921 ssarlash@purdue.edu
James Erwin VIP 47920 erwin17@purdue.edu	Aditya Hegde Volunteer hegde39@purdue.edu	

Abstract

In the rapidly evolving climate of AI research, Domain-Specific Accelerators (DSAs) must stay programmable to be able to adapt to evolving workloads without requiring costly hardware redesigns. This paper aims to enable high-level instruction of the Atalla Ax01 tensor core, an AI-focused DSA, using a compiler. There exists a wide selection of existing open-source compilers; however, none target Atalla’s ISA, nor alleviate key control flow responsibilities from the hardware, such as instruction scheduling and discovering instruction level parallelism in a way suited for Atalla. Hence, we propose **AtallaCC**, an extension of PPCI (Purely Python Compiler Infrastructure), which adds a custom AtallaC frontend to naturally reflect the behavior of the hardware, a backend architecture capable of targeting the Atalla ISA, discovering and organizing instruction level parallelism, and scheduling instructions to the hardware accelerator units provided by the Atalla DSA. These additions allow us to move the burden of instruction scheduling and dependency detection from the hardware to the compiler, substantially reducing hardware complexity, area, and cost. The resulting system is capable of emitting the entire Atalla ISA both as assembly files, and as assembled and linked ELF (Executable and Linkable Format) files, as well as performing correct instruction scheduling and VLIW (Very Long Instruction Word) packetization. Kernels written in AtallaC have been verified on the Atalla functional simulator, although testing and comparison with hand-written kernels revealed presence of poor memory utilization, redundant instructions, and insufficient VLIW slot usage. One limitation of this implementation is the usage of PPCI, which remains in active development, thus missing certain optimizations and requiring frequent corrections to the infrastructure outside of our implementation. Future work includes adding optimizations for tighter VLIW packing, less redundant instruction emission, and better memory usage, as well as staying compliant with evolving Atalla hardware. Overall, the finished compiler enables correct, efficient, and scalable high-level AI kernel development for Atalla hardware.

1 Introduction and Background

1.1 Problem Statement

Domain-specific accelerators (DSA) must be programmable to be able to adapt to evolving workloads without requiring costly

hardware redesigns. The Atalla Ax01 tensor core is one such DSA, specifically designed to target efficient AI acceleration workloads, such as general matrix multiplication (GEMM) and convolution. To enable programmability of the core, an instruction set architecture (ISA) was designed by the development team; however, instructing the core by writing assembly code using this ISA is slow, error-prone, and can be inefficient for large, complex workloads. A logical solution is to use a compiler to translate human-readable programs written in a higher-level programming language into the hardware-specific ISA. Additionally, due to the highly parallel nature of AI acceleration workloads, instructions must be scheduled into Very Long Instruction Word (VLIW) packets. A compiler is able to address this requirement by integrating instruction packetization algorithms into the code generation phase, essentially removing the burden of instruction scheduling and dependency detection from the hardware, and placing it onto the compiler. Overall, the compiler must produce correct assembly for verification on a functional simulator, as well as correct assembled and linked executable files for hardware execution and long-term usage.

1.2 Background

Before we describe the specific requirements for the design of the compiler, this section will introduce the foundational concepts referenced throughout the paper, including compiler architecture, the PPCI framework, and Very Long Instruction Word (VLIW) design.

1.2.1 Compilers Overview

A compiler is a program that translates source code written in a high-level language into a lower-level representation, typically machine code or assembly. The process begins with lexical analysis, where the source text is broken into tokens (keywords, identifiers, literals), followed by parsing, which organizes those tokens into an abstract syntax tree (AST) that reflects the grammatical structure of the program. Next, semantic analysis checks for type correctness and scope rules, annotating the AST with meaning. Together, these stages constitute the compiler’s “frontend,” which is largely language-dependent.

The “backend” then takes over, starting with intermediate code generation — producing a language- and platform-agnostic intermediate representation (IR). An optimizer ap-

plies transformations to this IR to improve performance or reduce code size, using techniques like dead code elimination, loop unrolling, and inlining. Finally, code generation maps the optimized IR to target machine instructions, handling register allocation and instruction selection. Some compilers also contain a linking stage that resolves references across separately compiled modules, producing a final executable.

1.2.2 Pure Python Compiler Infrastructure (PPCI)

Pure Python Compiler Infrastructure (PPCI), created and maintained by Windel Bouwman [1], is a compiler infrastructure written entirely in Python, with existing frontends for C and other languages. It also contains backend support for RISC-V, x86.64, ARM, etc. It is important to note that, as PPCI is a pre-production software, a number of bugs may still be present in its codebase, which may require allocating effort into alleviation. PPCI's optimization passes are also less mature than those of production-level compilers, such as LLVM [2] or GCC [3]. PPCI's user base is also much smaller, providing less developer support. Nonetheless, PPCI offers a realistically readable codebase, ease of modification and extension suitable for our two-semester timeline, which we believe makes it the appropriate foundation for Atalla.

1.2.3 VLIW

Very Long Instruction Word (VLIW) is a processor architecture designed to exploit Instruction Level Parallelism (ILP) by moving the task of instruction scheduling from the hardware to the compiler. In a traditional out-of-order superscalar processor, dedicated hardware logic dynamically checks dependencies and schedules parallel execution at runtime, adding significant area and power overhead. In contrast, a VLIW processor executes a fixed packet of independent instructions in parallel, with all scheduling performed by the compiler. This architecture is described in Josh Fisher's original VLIW proposal [4]. This design eliminates the need for runtime dependency-checking hardware, resulting in a simpler, more power-efficient design, at the expense of higher compiler complexity [5]. For AI acceleration workloads, which are compute-heavy and exhibit predictable instruction patterns that a compiler can schedule effectively, this trade-off is particularly well-suited.

1.3 Requirements

1.3.1 Creating an Atalla-Specific Compiler

Since hardware-level operation demands a low-level target language, C was initially chosen as the frontend language; however, Atalla-specific workflows require several modifications beyond what standard C provides. As a baseline, a compiler targeting a C-based frontend must handle global and local scalar (`int`, `char`) and aggregate (`array`, `struct`) data types, control flow constructs (`if-else`, `for`, etc.), function calls with proper calling conventions, and pointer/memory operations – all of which serve as the foundation before introduc-

ing Atalla-specific frontend extensions. In the backend, the compiler must target the Atalla ISA – a custom architecture, loosely based on the RISC-V Vector Extension (RVV), as defined in Atalla vector core specification [6]. However, Atalla contains several major vector changes in order to adapt this robust and widely-used architecture to Atalla workflows.

To enable efficient execution of AI acceleration workflows, such as matrix multiplication, Atalla performs operations on 512-bit "vector" registers, which consist of 32 16-bit brain floating point (BF16) values, and are designed to store an entire row of a matrix. Atalla also contains "mask" registers – these are 32-bit registers with one bit per BF16 value, used with every vector instruction to specify which values are used in the operation [6]. These hardware details impose additional requirements on both the frontend and backend. The frontend must allow the developer (end user of the compiler) to declare vector-type variables and invoke vector operations – ranging from simple arithmetic to more complex ones like GEMM – with appropriate mask values, all within the high-level C-based code. On the backend, the compiler must support vector and mask registers, vector memory management via the scratchpad, and BF16 arithmetic including type conversion between BF16 and integer types.

1.3.2 VLIW Packetization

While VLIW simplifies hardware design, it introduces significant complexity into the compiler backend. The compiler needs to implement a "packetization" pass to transform a linear stream of instructions into VLIW "packets" required by the Atalla core.

- **Structural Hazard Prevention:** The packetizer must map instructions to specific functional units used by said instruction and ensure no structural hazards. A structural hazard occurs when two instructions in the same packet require the same functional unit, making it impossible to execute them in parallel regardless of their data independence.
- **Data Hazard Detection:** The compiler must guarantee that instructions within a single packet are mutually independent to ensure they can execute in parallel. To do this, the compiler must analyze the Data Dependency Graph to prevent Write After Read (WAR) hazards that the hardware scheduler cannot internally resolve. Additionally, even though the hardware scheduler is able to internally resolve Read After Write (RAW) and Write After Write (WAW) hazards, doing so introduces delays. Therefore, the compiler will aim to prevent RAW and WAW hazards as well.
- **Code Density Optimization:** To maximize execution efficiency, the packetizer needs to fill as many of the four packet slots as possible.
- **Latency Minimization:** The packetizer must account for the different instruction latencies. The main goal of

the packetizer is to minimize total program latency by discovering and leveraging ILP.

1.3.3 Linking

Traditionally, it is the job of a linker to combine all of the assembled object files into a final executable container format. This executable binary file is then converted into a specific binary that the hardware can use to execute a given program. Overall, it is imperative that the compiler possesses a working linker so that it is able to properly combine files into a comprehensive singular executable program.

1.4 Prior Design

A previous team attempted to build the Atalla compiler using LLVM but was unable to produce working assembly within the available timeline. The scale of LLVM’s frontend and backend infrastructure consumed most of the development window, which motivated the subsequent pivot to PPCI. Prior work within our current team established a partially working proof-of-concept Atalla compiler built on PPCI and demonstrated that both the frontend could support basic vector extensions and backend could be configured to support the scalar portion of the custom ISA [7].

2 Design Choices

2.1 Atalla Frontend & Backend

2.1.1 AtallaC Parser

We have established that a version of C, extended for Atalla-specific workflows will be used as our target frontend programming language. We have called this extended language option **AtallaC**. Our first major decision regarding the AtallaC parser was not to write one from scratch. Considering that PPCI already includes a fully-functional C parser [1], copying and extending it will allow us to save development time and debugging effort, enabling us to focus only on the additions/changes.

In the following sections, we will discuss further additions to the AtallaC parser that involved major design decisions, as well as their implications for the backend implementation.

2.1.2 BF16 Implementation

The existing C implementation used basic 32-bit floats as its floating-point datatype. As Atalla does not support 32-bit floats, and instead uses BF16, we decided to remap the `float` datatype keyword to emit a new `ir.bf16` IR datatype instead of the prior `ir.f32` IR datatype, which will then be used to inform IRDAG node generation for floating-point operations. An alternative was to instead create a *new* keyword (e.g. `bfloat16`) to map to the new IR type. This decision turned out to be slightly controversial, as some believed that silently remapping the `float` keyword to a different datatype could be confusing to a developer who may be expecting a 32-bit

float to be used when invoking `float`, but accidentally ends up using BF16. However, we believe that a developer should be aware that the only floating-point variable type available to Atalla is BF16, and thus maintaining one universally recognized keyword would be a more accessible solution. This does not have any direct impact on compilation or efficiency, rather being an open discussion about user experience.

2.1.3 BinOp Usage & Intrinsic Functions

The developer can invoke most of the instructions in the Atalla ISA using binary operators (BinOps). For example, if one tries to add together two integers, it can be done as simply as:

```
int a, b;
...
int c = a + b;
```

The same applies to vectors instructions, a large portion of which can also be invoked with BinOps. For example, to multiply two vectors:

```
vec va, vb;
...
vec vc = va * vb;
```

Some operations, however, such as SQRT, do not have an associated BinOp; another example is GEMM, which requires three operands and is therefore incompatible with binary operators. The least implementation-heavy way to allow the developer to invoke such instructions is to use inline assembly (ASM), as the logic for it already exists in PPCI. This would require a developer to directly invoke the desired assembly instruction using this structure.

A better, more implementation-heavy solution is to create a set of intrinsic functions. These are a special type of function recognized directly by the compiler, which substitutes the call with an architecture-specific sequence of instructions rather than generating a standard function call. They are commonly used to expose low-level hardware operations directly to the programmer without requiring inline assembly. In addition to reducing complexity for the programmer, intrinsic functions should also result in more efficient generated assembly. The reason for this is that inline ASM must remain abstract as a measure against unknown assembly behavior. In contrast, the compiler is aware of the specific behavior caused by each intrinsic function, removing the need for the safety measures and allowing for instruction-specific optimizations [8].

Due to the reduced programming complexity and positive effects on assembly efficiency, we decided to proceed with implementing intrinsics for all instructions that do not have a corresponding BinOp or are incompatible with BinOps entirely.

2.1.4 Mask Datatype

As mask registers are a separate register type from scalar registers, the initial idea was to create a new frontend datatype (e.g. `mask`). However, after evaluating the ways masks are used

in workflows heavily featuring vector instructions, we decided not to expose masks as a separate datatype to the developer. There were two main factors from this analysis that informed this decision.

First, vector operations invoked via BinOps do not need an explicit mask value, as using a BinOp implies that the programmer wishes to perform the element-wise operation on all elements of the two vectors. However, the instruction still needs a mask register. For this purpose, we can use a special mask register in Atalla hardware, which is hardwired to `0xffffffff` (32 1's in binary), meaning the vector operation would be performed on all elements. Thus, when emitting assembly from vector instructions invoked via BinOps, the compiler can automatically emit the special mask register, without the need for explicit specification.

Second, for cases where the developer does need a specific mask, we can introduce an intrinsic function that can accept an integer as a substitute for a mask, then emit a typecast instruction to place said integer value into a mask register. This would result in both smoother, less confusing development and more efficient instructions, as relying on mask creation instructions (e.g. `MVV`) for all mask creation can be more instruction-heavy than simply converting an integer. Mask creation will still be supported using intrinsics: in cases where a developer assigns the return to an integer, a typecast instruction will also be emitted.

This concludes the design decisions made in the frontend. We will now discuss the design choices made in the backend implementation of the Atalla architecture.

2.1.5 Scratchpad Stack

As vector registers were to be added to the backend of the compiler, we considered ways to handle vector register spills. A spill occurs when the register allocator runs out of physical registers and is forced to store a variable's value to memory (the stack) temporarily, reloading it later when needed. Furthermore, in order to enable pointer operations on vector-type variables, they must be stored to memory. Scalar registers are able to be stored to the stack, which is an allocated memory space in DRAM. Vector data, however, is stored in the scratchpad – an on-chip SRAM memory unit designed for tile storage, as described in Atalla's memory system design [9]. This unit initially did not have a dedicated space for a stack. Instead, both developers and the compiler would have had to specify an exact address in scratchpad memory to store the vector.

This presented two options from the standpoint of the compiler. The easier option would be to disallow vector spills in the compiler, throwing an error whenever running out of vector registers. For pointer operations, this would mean that the compiler must resolve an exact scratchpad memory address to store each vector register, which would necessitate complex memory tracking logic as there would have been no dedicated memory space for temporary storage. The more involved solution was to create logic for a *scratchpad stack* in the Atalla architecture. This, however, required us to communicate the need for this reserved scratchpad memory space and for the

scratchpad team to implement this suggestion.

After receiving approval from the scratchpad team that such a stack would be possible, we decided to proceed with implementing the scratchpad stack into the compiler, in order to allow safe and consistent temporary storage of vector registers, enabling elegant spilling and pointer logic – a key baseline goal of the compiler.

2.2 VLIW Packetization

2.2.1 Algorithm Selection

A Greedy List Scheduling algorithm was inherited from a previous iteration's packetization algorithm development [7]. Our main design choice was whether to proceed with integrating this algorithm into the compiler or to create a more optimal but slower and vastly more difficult global scheduling method. We decided to proceed with list scheduling due to greater computational efficiency, simplicity, and scalability, as well as this algorithm's status as the industry standard [10]. The main trade-off is that this may result in lower code density compared to the global scheduling.

2.2.2 Control Flow and Branching

In a VLIW architecture, branch instructions inherently complicate hardware design because they disrupt the deterministic, parallel execution. If a branch is issued simultaneously alongside other operations, the hardware must incorporate complex dependency checking, branch prediction, and multiplexing logic to safely resolve the unpredictable change in the execution path. To alleviate this burden on the hardware, the compiler enforces a single-issue branch policy. By isolating any control flow instruction into slot 0 and padding the remainder of the VLIW packet with NOPs, the hardware is guaranteed stable behavior during branches and jumps. Although this will result in slightly reduced VLIW slot utilization in branch-heavy programs, the significant hardware complexity alleviation justifies this decision.

2.2.3 Integration into AtallaCC

To integrate the developed packetization algorithms into AtallaCC, we decided to implement a post-scheduling and post-register-allocation pass, as the inherited algorithm acted on previously register-allocated and scheduled assembly code [7]. It was not possible to integrate this algorithm elsewhere in the compiler without reworking major parts of the algorithm to handle either register allocation, scheduling, or potentially both. Thus, we will rely on PPCI's implementation of said algorithms when producing our VLIW packetization.

2.3 Global Variables & Linking

The PPCI linker by default outputs a binary ELF (Executable and Linkable Format) file. The ELF file format is simplistic, making it easy for the Atalla hardware to parse and utilize.

Hence, we chose to proceed with this format as the executable output required of AtallaCC.

The binary object files returned by the assembler often contain unresolved address references, often referred to as symbol values. These symbols are placeholders for addresses that may point to function labels not contained in a given file, global variables that are declared or modified in multiple files, and other data structures. The linker must resolve all symbol values to specific memory addresses. This is done by relocations, which is where the linker assigns the variable a correct memory address to replace the symbol value [11].

Initially, AtallaCC did not possess the ability to relocate any symbol values, and therefore could not combine all of the assembled object files into one ELF file. To solve this issue, we need to implement relocations for all of the applicable Atalla instructions. We also need to create specific relocation functionality for global variables and enable linking to the ELF file format.

It is important to have a method of verifying that the program contained in the ELF binary is indeed the program that was originally written. Hence, we chose to build a disassembler which takes as input an ELF file, and outputs an assembly-like file. This resulting file can be used to verify that the program in the ELF file is consistent with the AtallaC program.

3 Architecture

We have made four major additions to PPCI in order to create AtallaCC. These are the frontend, backend, VLIW packetizer, and relocation classes to enable linking. These new modules interact with existing PPCI infrastructure to produce a fully functioning compiler. A summary of how our new additions affect code- and data-flow through the compiler can be seen in figure 1. Here, the custom AtallaC frontend provides semantics to the PPCI parser, which generates the IR. The CodeGen phase structures IR into an IR Directed Acyclic Graph (IRDAG), then uses the IRDAG-to-ISA mapping provided to it by the Atalla Arch Backend to generate assembly, which can optionally be returned by the compiler at this stage. After assembly is generated, it is passed through to the VLIW packetizer, emitting packetized assembly. The assembler accepts assembly code (whether packetized or not) and encodes it into binary format object files, using encodings provided by Atalla Arch. Finally, the Linker uses relocation classes specified in the Atalla Arch to combine object files and resolve references between them, generating a final ELF file.

In this section, we will discuss the implementation nuances involved in configuring each of the major additions we made to PPCI to create AtallaCC.

3.1 AtallaC Frontend Architecture

As mentioned previously, to create the AtallaC frontend, we copied the existing PPCI C frontend, which served as a source of baseline C functionality. Here, we made two major additions: the new datatypes and the intrinsics.

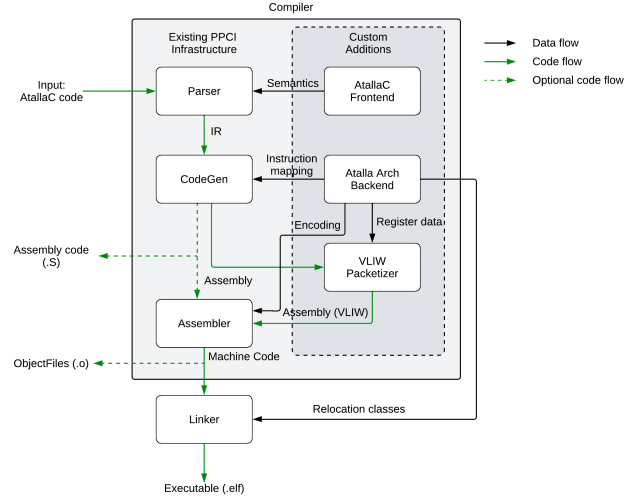


Figure 1: AtallaCC extends PPCI at four points in the pipeline, isolating Atalla-specific additions from existing infrastructure.

3.1.1 Datatype Changes

First, we added the `vec` datatype to the list of available type specifiers that the AtallaC parser can recognize. In order to enable IR generation for the `vec`, `bf16`, and `mask` datatypes, we defined new IR datatypes. Note that the `mask` datatype will not be exposed to the list of available type specifiers and simply exists to generate type conversion (`mask` to integer and vice-versa) IRDAG nodes.

To enable AtallaCC to map datatype keywords to IR variable types, the AtallaC IR Code Generator requires an IR type map. Complementing the basic C mapping inherited from the C frontend, vector and mask type mappings were also added. Here, `float` keyword was remapped from 32-bit floats to instead generate BF16 IR variables.

3.1.2 Intrinsic Function Implementation

To implement intrinsic functions, the entire stack of the compiler was modified. The frontend was modified at three stages to support this. First, the keyword is registered in the parser and a grammar rule is defined to recognize and verify the intrinsic call syntax. Then, the parser invokes the semantics layer, which performs type checking on all operands and constructs an AST node representing the operation. From the AST, the IR code generator evaluates each argument and calls the IR builder to emit a corresponding IR instruction, which carries operand and type information through to the backend. In the backend, a DAG selection function converts the IR instruction into a named IRDAG node, which the instruction selector then matches against a backend ISA pattern to emit the final machine instruction. This pipeline, spanning 11 files across all compilation layers, is consistent for all intrinsics, making the addition of new operations a well-defined process. By following this pipeline, intrinsics were created for all instructions with no way of direct invocation using BinOps. A

list of added intrinsics can be found in the README of the AtallaCC repository (Appendix B).

3.2 Atalla Backend Architecture

Due to Atalla’s similarity to RISC-V and RVV in its scalar and vector implementation respectively, the RISC-V architecture was copied into a new `atalla` subdirectory to be used as the basis for the Atalla architecture. First, all instruction classes (e.g. R, I, VV, MVS, MTS, etc.) were created. These classes contain the information used by the assembler to correctly encode the assembly instructions into binary code. Each individual instruction is an instance of one of these classes. Then, the scalar instruction file was adapted to instantiate the correct instructions using opcodes for each instruction provided by Atalla’s ISA. This file was also extended with ISA patterns to support BF16-type IRDAG nodes. A vector and mask register file was added to specify the list of available hardware registers. Finally, we created a new vector instructions file to instantiate all vector instructions, their opcodes, and the necessary ISA patterns.

3.2.1 Scratchpad Stack Implementation

The scratchpad stack was implemented by extending the backend `Frame` class that PPCI uses to handle stack and frame management. To the `Frame` class, we added a “stack type”, an enum value used to distinguish whether the allocation/retrieval command the `Frame` class receives concerns the DRAM stack or the scratchpad stack.

The backend architecture was configured to reflect this by reserving registers `x32` and `x33` for the frame and stack pointers respectively. As `x33` is not hardwired to the stack address, the compiler will load the scratchpad stack address into this register in the prologue of the `main` function that each program contains. To enable proper usage of the frame pointer, the compiler will automatically load the scratchpad stack address, plus the required offset into `x32`. Together, these additions allow the compiler to generate loads and stores to and from addresses in the scratchpad stack, correctly handling spills, pointer logic, and function calls.

3.3 Disassembler and Relocation Implementation

As established in section 2.3, relocation classes were added to Atalla Arch Backend for all instructions that required it to enable linking. These include `jal` and `BR`-types for control-flow; `lui` and `addi` for loading global variable addresses.

With relocations implemented, the ELF output could now be generated. These files consist of several sections, such as `.text`, `.data`, and `.bss` sections as well as the symbol table. The locations of these sections are stored in the ELF header. Program instructions are stored in `.text`, and any other data or global variables are stored in the `.data` section. The symbol table describes the locations of all functions and global variables in memory. Additionally, the starting addresses of the `.text` and

`.data` can be controlled via a memory map which can be specified by the programmer.

To verify the ELF file, the disassembler parses the ELF header to determine the locations of the sections above. After this, the disassembler parses the rest of the file and outputs an assembly-code-like `disassembly.txt` output file that contains a table with the addresses of the global variables, all of the symbols from the symbol table and their respective addresses, and the disassembled code for further verification.

3.4 VLIW Packetizer Implementation

The VLIW packetizer was implemented as a core component of the `CodeGenerator` class. It is positioned after register allocation but before the final assembly emission in order to allow later assembling and linking of the code, not just packetized assembly output. The implementation is centered around the `pack_flat_vliw` method, which performs a packetization on the previously linear instruction stream. To ensure compliance with established requirements the packetizer maintains the following information:

- **Occupied Functional Units:** A set of hardware units that are already assigned in the current packet.
- **Vector Lane Count:** A counter to enforce the maximum of two vector operations per packet, as per hardware lane availability.
- **Register Hazards:** A map of registers that are being written to and read from in the current cycle to prevent RAW, WAR, and WAW hazards within a single packet.

4 Results

4.1 Compiler Validation Testing

To verify correctness of AtallaCC, we developed a dedicated validation suite targeting major features of the compiler. The goal was not simply to confirm that each test compiled and manually inspect the output, but to automatically verify that the generated code produced the correct result when executed on the Atalla functional simulator, which was provided by the Systems Software team. The test suite consists of small, focused AtallaC programs, where each file exercises one compiler feature, and files that combine these features together. Each validation file was designed with a known input and a known expected output. Additionally, to validate compiler functionality on real AI workflows, the suite was extended with AtallaC versions of AI computation kernels, such as GEMM, convolution, softmax, flash attention, and others. The code for these kernels was also provided to us by the Systems Software team.

4.2 CI Pipeline

To make validation repeatable and automatic, we configured a GitHub Actions workflow. The workflow runs whenever a

pull request is opened, reopened, marked ready for review, or updated with new commits. The CI job runs both the basic test suite and the kernel suite. On failure, the workflow uploads the validation artifacts, including simulator outputs and logs, so that the failing test can be debugged directly from the CI run.

4.3 Kernel Performance Evaluation

To evaluate the compiler-generated code, a set of AI kernels was compiled from AtallaC and executed on the Atalla functional simulator. To enable baseline comparison, hand-written assembly versions of these kernels were also executed on the simulator. Note that this is not a cycle-accurate simulator, so collected data is restricted to correctness and instruction/packet execution counts. Results were compared across the following simulator-derived metrics and counters: cosine similarity of output (a measure of vector similarity to theoretical output), absolute maximum error (largest difference between any single output value), Floating Point Operations or FLOPs (F) – denoting useful arithmetic operations, but including redundant operations, bytes written to (B_{wr}) and loaded from (B_{ld}) memory, instructions executed (I), and packets executed (P). From these results, we also calculated average VLIW slot utilization (η), and arithmetic intensity (α). The calculation of these metrics is done by using equations (1) and (2) respectively. In equation (1), 4 represents the number of instructions per packet – for Atalla, this number is set at 4.

$$\eta = \frac{I}{4 \times P} \quad (1)$$

$$\alpha = \frac{F}{B_{ld} + B_{wr}} \quad (2)$$

A kernel with high arithmetic intensity does more computation per byte transferred, meaning it is likely to be limited by the speed of the compute units rather than the speed of memory – this is called being compute-bound. A kernel with low arithmetic intensity does relatively little computation per byte transferred, meaning it spends most of its time waiting for memory – this is called being memory-bound. Under the roofline model [12], compute-bound operation is the desirable regime for an AI accelerator, as it ensures the available floating-point units are fully utilized rather than sitting idle waiting for data.

4.3.1 Numerical Correctness

With the exception of `flash_attention`, the compiler achieves cosine similarity of 0.9999 or higher across all kernels, matching the hand-written reference. The `flash_attention` kernel reaches a cosine similarity of 0.977 compared to 1.0 for the hand-written version. For `layernorm`, the compiler output is numerically closer to the theoretical result than the hand-written assembly, with a maximum error of 0.016 compared to 1.77 in the hand-written version.

4.3.2 Instruction Count & Slot Utilization

For small and medium kernels, the compiler emits substantially more instructions than the hand-written reference. Table 1 shows representative cases. The most significant overhead occurs in `flash_attention`, where the compiler emits over 10× more packets and 8× more operations. For large GEMM workloads (512^3 , 1024^3), packet counts are slightly lower for the compiler (−11% to −14%), though operation counts remain slightly higher (+3% to +8%). `maxpool` is the closest match, with effectively identical counts across both metrics.

Slot utilization on reduction-heavy kernels is notably lower: `softmax` (0.362 vs. 0.432, −16%), `attention` (0.372 vs. 0.443, −16%), `flash_attention` (0.322 vs. 0.371, −13%), and `relu` (0.390 vs. 0.414, −6%). On kernels such as `maxpool`, `conv`, and all GEMM variants, the compiler matches or slightly exceeds hand-written slot utilization.

4.3.3 Memory Transfer Volume and Arithmetic Intensity

For elementwise and convolution kernels, total memory transfer volume is identical between the compiler and hand-written implementations. For `layernorm` and all GEMM variants, the compiler transfers substantially more bytes in total. Table 2 shows the breakdown for selected kernels. For GEMM, the excess is concentrated in writes, which grow significantly with problem size while read volume remains identical (512^3 , 1024^3), write overhead reaches +1500% and +3100% respectively with no corresponding increase in reads. For `flash_attention`, the pattern is reversed: the compiler writes fewer bytes (−67%) but loads substantially more (+984%), resulting in a +634% increase in total transfer volume. For `layernorm`, both reads and writes are +700%. This excess transfer volume directly reduces arithmetic intensity across all affected kernels. For all GEMM variants, AI is 27–34% lower in the compiler output than in the hand-written reference. For `flash_attention` and `layernorm`, AI is reduced by 86% and 87% respectively.

5 Discussion of Results

5.1 Where the Compiler Works Well

The compiler’s strongest results occur on `maxpool` and `conv`, where instruction counts, slot utilization, and arithmetic intensity all closely match the hand-written reference. Numerical correctness is strong across the board, with cosine similarity of 0.9999 or higher on all kernels except `flash_attention`. Notably, for `layernorm`, the compiler produces a more numerically accurate result than the hand-written reference. These results confirm that AtallaCC’s core instruction emission and packetization pipeline are correct, and that the gaps observed elsewhere are not caused by fundamental correctness problems in the backend.

Table 1: Packets executed, operations executed, and slot utilization for selected kernels: compiler (C) vs. hand-written (HW).

Kernel	P (C)	P (HW)	ΔP	I (C)	I (HW)	ΔI	$\Delta \eta$
relu_4x32	78	26	+200%	108	43	+151%	-6%
softmax_row32	49	22	+123%	70	38	+84%	-16%
flash_attention_n32_d32	18,405	1,618	+1038%	20,979	2,431	+763%	-13%
maxpool_8x8	362	365	-1%	534	534	0%	+2%
gemm_512x512x512	1,706,946	1,915,495	-11%	2,239,066	2,071,929	+8%	+9%
gemm_1024x1024x1024	13,264,210	15,313,063	-13%	17,389,890	16,561,337	+5%	+12%

Table 2: FLOPs, Memory transfer overhead and arithmetic intensity for selected kernels: compiler (C) vs. hand-written (HW).

Kernel	ΔB_{ld}	ΔB_{wr}	ΔB_{total}	ΔF	α (C)	α (HW)	$\Delta \alpha$
layernorm_4x4	+700%	+700%	+700%	+3.6%	2.18	16.86	-87%
flash_attention_n32_d32	+984%	-67%	+634%	+2.4%	1.10	7.88	-86%
gemm_32x32x128	+33%	+300%	+60%	+6.2%	8.65	13.03	-34%
gemm_64x64x256	+18%	+700%	+56%	+5.1%	9.77	14.47	-32%
gemm_512x512x512	0%	+1500%	+44%	+4.3%	11.08	15.32	-28%
gemm_1024x1024x1024	0%	+3100%	+47%	+4.2%	11.19	15.79	-29%

5.2 Where the Compiler Struggles

5.2.1 Instruction Overhead

The excess instruction counts on small and medium kernels are a known consequence of PPCI’s code generation, which emits instructions that a more mature optimizer would combine or eliminate. Despite emitting substantially more packets, the compiler achieves only slightly higher total FLOPs than the hand-written reference, indicating that the excess packets consist largely of non-compute instructions rather than additional arithmetic work. This overhead is most visible at small problem sizes and becomes negligible at large GEMM scales. The issue is therefore more significant for lightweight kernels common in inference workloads than for the large matrix operations that dominate training.

5.2.2 VLIW Slot Utilization

The slot utilization gap on several kernels is attributable to PPCI’s existing scheduler serializing operations that could be executed in parallel. This is a direct limitation of the VLIW packetizer, which operates on a linear post-register-allocation instruction stream and cannot re-allocate registers to re-parallelize previously serialized instructions.

5.2.3 Memory Transfer Volume

The excess memory transfer volume on GEMM, layernorm, and flash_attention is the most impactful unresolved issue. The nature of the excess differs by kernel: for GEMM it is concentrated in writes, which grow with problem size while read volume remains identical to hand-written; for flash_attention the compiler loads substantially more bytes than hand-written while writing fewer; for layernorm both reads and writes are elevated. In all cases the effect is the same — arithmetic intensity is reduced, pushing these kernels toward the memory-bound

regime on hardware designed to be compute-bound. The root cause has not yet been diagnosed.

5.2.4 Flash Attention Accuracy

The numerical gap in flash_attention is an isolated issue with no diagnosed cause. Given that all other kernels meet or exceed hand-written accuracy, it does not indicate a systemic correctness problem, but it does mean the compiler cannot yet be considered fully validated for this kernel.

5.3 Overall Assessment

The compiler meets its primary goal of producing correct, executable code for the Atalla ISA without requiring manual instruction writing or hardware scheduling. The identified gaps between compiled and hand-written kernels are specific and bounded: instruction overhead from PPCI’s code generation, underutilized VLIW slots from serialized parallelizable operations, excess memory transfer volume on several key kernels, and one unresolved numerical accuracy issue. None of these represent fundamental correctness failures, and all are identifiable targets for future optimization work.

5.4 Validation Discussion

Coverage is currently defined by the set of intrinsics and instruction patterns we chose to exercise. The suite does not claim exhaustive coverage of the Atalla ISA, and some behavior outside the tested paths has not been validated. Correctness is also validated against the functional simulator rather than real hardware, so any discrepancy between the simulator and the final hardware would not be caught by this suite.

6 Limitations

The largest limitation of AtallaCC is the usage of PPCI as the basis, which remains in active development, and has not reached a production level of maturity. The unfinished status of the infrastructure outside of Atalla-specific additions may introduce bugs in some untested code paths. Additionally, some present algorithms such as scheduling and register allocation may not be producing code that is efficient for Atalla’s use-case, a limitation we were not able to address.

7 Future Work

Future work will consist in large part of compiler optimizations. In order to bring the operation and packet counts up to par with or potentially improve on handwritten kernels, we must add several additional optimization passes, including loop optimization techniques (e.g., unrolling, fusion, invariant code moving), more robust peephole optimization to remove redundant instructions, and inline expansion to allow function inlining. In order to address VLIW slot under-utilization and improve overall latency, the compiler will need to decouple PPCI’s linear instruction scheduling from packetization passes, potentially requiring a custom register allocation and instruction scheduling algorithm. Crucially, the compiler must continue to adapt to evolving hardware requirements, such as ISA changes or register banking needs.

8 Conclusion

This paper presented AtallaCC, a compiler stack for the Atalla AI accelerator built as an extension of PPCI. Starting from a partially working scalar compilation path, we added full vector and BF16 frontend support, a scratchpad stack for safe vector register spills, intrinsic functions for the full Atalla ISA, an integrated VLIW packetizer, and a complete linking pipeline with relocation support and an automated disassembler for verification. The result is a compiler that accepts AtallaC source code and produces correct, assembled, and linked ELF executables without requiring the programmer to write assembly manually, and, most importantly, alleviating hardware scheduling requirements, significantly reducing hardware complexity. The primary gaps, concerning underutilized VLIW packets, redundant instructions, and poor memory usage, are all traceable targets for future optimization rather than fundamental correctness failures. Future work will focus on decoupling the inherited linear scheduler from the packetization pass to enable true instruction-level parallelism and adding further optimization passes. The compiler will also need to continue adapting to evolving ISA requirements as the Atalla hardware continues to mature.

References

- [1] W. Bouwman, “PPCI Documentation.” [Online]. Available: <https://ppci.readthedocs.io/en/latest/>
- [2] “LLVM Documentation.” [Online]. Available: <https://llvm.org/docs/>
- [3] “GCC Documentation.” [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc-15.2.0/gcc/>
- [4] J. A. Fisher, “Very Long Instruction Word architectures and the ELI-512,” *SIGARCH Comput. Archit. News*, vol. 11, no. 3, p. 140–150, Jun. 1983. [Online]. Available: <https://doi.org/10.1145/1067651.801649>
- [5] S. Alkhalefah and K. Aurangzeb, “Architectural comparison of vliw and superscalar processors,” in *8th International Conference on Computing, Control and Industrial Engineering (CCIE2024)*, Y. S. Shmaliy, Ed. Singapore: Springer Nature Singapore, 2024, pp. 147–155.
- [6] J. Ghanem, N. Vaidyanath, J. Walter, C. Johnson, and V. Sharma, “Vector Data Processing and GEMM Scheduling within the Atalla Ax01 Processor,” *Purdue SoCET Wiki*, 2025.
- [7] H.-I. Chu, M. Golovenchits, and S. Patel, “AI Hardware Compiler: A VLIW Approach to Instruction Scheduling for Accelerator Architectures,” *Purdue SoCET Wiki*, 2025.
- [8] Microsoft, “Compiler intrinsics,” learn.microsoft.com, 08 2021. [Online]. Available: <https://learn.microsoft.com/en-us/cpp/intrinsics/compiler-intrinsics?view=msvc-170>
- [9] A. R. Ravikiran, J. Hernandez, H. Kwon, and R. M. M. Pinheiro, “On the Design of a Software-Controlled Memory System for the Atalla Ax01 Artificial Intelligence Accelerator,” *Purdue SoCET Wiki*, 2025.
- [10] K. D. Cooper and L. Torczon, “Instruction scheduling,” *Elsevier eBooks*, pp. 617–662, 09 2022. [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/instruction-scheduling>
- [11] M. Mishra, *Relocations: How the Linker Patches Your Code*. Berkeley, CA: Apress, 2026, pp. 107–121. [Online]. Available: https://doi.org/10.1007/979-8-8688-2154-7_6
- [12] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

A Teamwork

Work was divided between teammates mainly by interest and expertise, though sometimes team member were assigned to tasks as required by urgency. A summary of contributions by each team member can be found in table 3. For the report, each team member was responsible for writing up their part of the work on the project itself – for all major sections of the report. Additional contributions on the report from each member are also listed.

Team Member	Contribution to Project	Additional Contributions to Report
Mikhail Golovenchits	Project Lead, Frontend, Backend	Abstract, Problem Statement, Compiler Overview, Results, Discussion, Future Work, Conclusion
Sahil Patel	Verification, CI/CD	Prior Design
James Erwin	Globals, Disassembler, Linker	
Shaunak Sarlashkar	VLIW Packetization	
Aditya Hegde	Miscellaneous	Limitations

Table 3: Team Member Contribution Summary

B Code Repository

The AtallaCC compiler source code and README are available at: <https://github.com/Purdue-SoCET/aihw-ppci-compiler>